

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Your Code

```

1 import java.util.*;
2 class Main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         sc.useDelimiter("[,\\s]+");
6         try{
7             String name = sc.next();
8             int age = sc.nextInt();
9             double temp = sc.nextDouble();
10            if(age<18){
11                throw new InvalidAgeException("InvalidAgeException");
12            }
13            if(temp <95.0||temp>105.0){
14                throw new InvalidTemperatureException("InvalidTemperatureException");
15            }
16            System.out.println("Patient registered sucessfully");
17        }catch(InvalidAgeException | InvalidTemperatureException e){
18            System.out.println(e.getMessage());
19        }catch(Exception e){
20            System.out.println("invalid input");
21        }finally{
22            System.out.println("Registration completed");
23            sc.close();
24        }
25    }
26 }
27 class InvalidAgeException extends Exception{
28     public InvalidAgeException(String s){
29         super(s);
30     }
31 }
32 class InvalidTemperatureException extends Exception{
33     public InvalidTemperatureException(String s){
34         super(s);

```

Input

Arun
35
98.6

Output

Patient registered sucessfully
Registration completed

Online Food Delivery - Multiple
Threads and Execution Order
10 marks

5/5 answered

Your Code

```
1 import java.util.InputMismatchException;
2 import java.util.*;
3 public class InsufficientBalanceException extends Exception{
4     InsufficientBalanceException(String s){ super(s); }
5 public static void main(String[] args){
6     Scanner sc = new Scanner(System.in);
7     try{
8         double bal = sc.nextDouble();
9         double amt = sc.nextDouble();
10        if(amt <= 0){
11            throw new IllegalArgumentException("Invalid amount");
12        }
13        if(amt>bal){
14            throw new InsufficientBalanceException("Insufficient bal
15        }
16        System.out.println("Remaining: "+(bal - amt));
17    }
18    catch(InputMismatchException e){
19        System.out.println("Invalid numeric input");
20    }
21    catch(Exception e){
22        System.out.println(e.getMessage());
23    }
24    finally{
25        System.out.println("Transaction completed");
26    }
27 }
28 }
```

✓ Submitted

Input

10000 3000

Output

Remaining: 7000.0
Transaction completed

Threads and Execution Order
10 marks

5/5 answered

Your Code

```
1 import java.util.*;
2 class Main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         sc.useDelimiter("[,\\s]+");
6         String[] products={"Laptop","Phone", "Tablet", "Headphone", "Ke
7         double[] prices={50000.0 , 20000.0, 15000.0, 2000.0, 1000.0};
8         try{
9             int index = sc.nextInt();
10            int quantity = sc.nextInt();
11            if(quantity <= 0){
12                throw new IllegalArgumentException("Invalid quatity");
13            }
14            double totalCost = prices[index]*quantity;
15            System.out.println("Total Cost: "+totalCost);
16
17        }catch(InputMismatchException e){
18            System.out.println("InputMismatchException Handled");
19        }catch(ArrayIndexOutOfBoundsException e){
20            System.out.println("ArrayIndexOutOfBoundsException Handled"
21        }catch(IllegalArgumentException e){
22            System.out.println("IllegalArgumentException Handled");
23        }finally{
24            sc.close();
25        }
26    }
27 }
```

Submitted

Input

1,2

Output

Total Cost: 40000.0

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

```

1 class Main{
2     public static void main(String[] args){
3         HospTask emergency = new HospTask("Emergency Patient Processing");
4         HospTask general = new HospTask("General Patient Processing");
5         HospTask billing = new HospTask("Billing Processing");
6         emergency.setPriority(Thread.MAX_PRIORITY);
7         general.setPriority(Thread.NORM_PRIORITY);
8         billing.setPriority(Thread.MIN_PRIORITY);
9
10        try{
11            emergency.start();
12            emergency.join();
13
14            general.start();
15            general.join();
16
17            billing.start();
18            billing.join();
19        }catch(InterruptedException e){
20            System.out.println("Thread executionn interrupted: " + e.getMessage());
21        }
22    }
23 }
24 class HospTask extends Thread{
25     private String taskName;
26     public HospTask(String taskName){
27         this.taskName = taskName;
28     }
29     public void run(){
30         System.out.println(taskName+ "-Priority: " + getPriority() + "is running");
31     }
32 }

```

stdin input

Output

```

Emergency Patient Processing-
Priority: 10is running
General Patient Processing-
Priority: 5is running
Billing Processing-Priority: 1is
running

```


5/5 answered

```

2 class Main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         String order = "Pizza", pay = "UPI", agent= "Agent A";
6         if(sc.hasNext()){
7             String s = sc.nextLine().replace(";", "");
8             for(String part: s.split("\\s+")){
9                 if(part.startsWith("Order=")) order = part.substring(6);
10                if(part.startsWith("Payment=")) pay=part.substring(8);
11                if(part.startsWith("Delivery=")) agent = part.substring(9);
12            }
13        }
14        try{
15            Task t1 = new Task("Order Processed: " + order);
16            t1.start();
17            t1.join();
18            if(pay.equalsIgnoreCase("Failed")){
19                System.out.println("Payment Status Failed");
20                System.out.println("Payment Failed. Delivery Skipped");
21                return;
22            }
23            Task t2 = new Task("Payment Successful via "+ pay);
24            t2.start();
25            t2.join();
26            Task t3 = new Task("Delivery Assigned to "+ agent);
27            t3.start();
28            t3.join();
29        }catch(InterruptedException e){
30            System.out.println(e.getMessage());
31        }
32    }
33 }
34 class Task extends Thread{
35     String msg;
36     Task(String msg){

```

Scanner Input

Output

```

Order Processed: Pizza
Payment Successful via UPI
Delivery Assigned to Agent A

```